



GREEDY 알고리즘 스터디, 숲속의 기린

다이나믹 프로그래밍 (1)



by 요술토끼(curious.wzrabbit@gmail.com)

다이나믹 프로그래밍이란 ✨

- ✓ 어떤 큰 문제를 해결하기 위해, 큰 문제를 작은 문제들의 연장선상으로 보는 기법입니다. 큰 문제를 종류가 같은 작은 문제들로 나누고, 이 작은 문제들을 이용해 큰 문제를 해결하고자 하는 발상입니다.
- ✓ 큰 문제에 대한 답을 구하기 위해 보통 작은 문제들에 대한 답을 이용해 식으로 도출해내게 됩니다. 이를 점화식이라고 합니다.

다이나믹 프로그래밍이란 ✨

- ✓ 그리고, 문제 해결 과정에서 구한 작은 문제들의 답을 저장해두었다가 재활용하는 기법입니다. 이를 통해 효율성을 극대화합니다. 이를 메모이제이션이라고 합니다.
- ✓ 이러한 값들은 흔히 n 차원 배열에 저장해 둡니다. 이후 이 배열에 있는 값을 재사용하기도 하고, 이 값들을 이용해 새로운 값을 배열에 저장하기도 합니다. 이 배열을 dp 테이블이라고 합니다. (dp = Dynamic Programming)
- ✓ 이 두 가지 기법을 사용하는 것이 다이나믹 프로그래밍의 핵심입니다.
- ✓ 또한, 다이나믹 프로그래밍은 “동적 계획법” 이라고도 불립니다. 둘은 동의어입니다.

다이나믹 프로그래밍이란 ✨

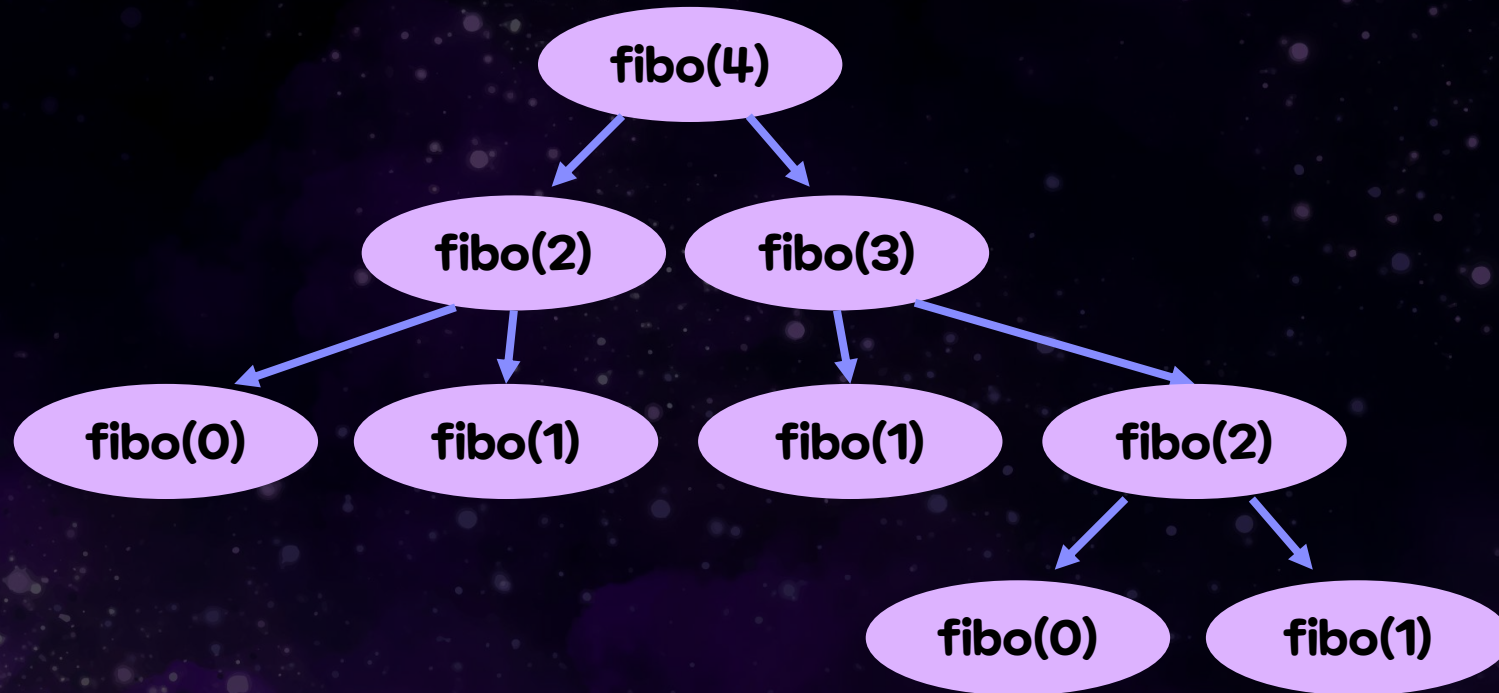
- ✓ 추상적이기도 하고, 생소하기도 할 것이라 생각합니다.
같은 요구사항이나 문제 제한조건이 다른,
문제셋의 A번 - 피보나치 수 5와 B번 - 피보나치 수 2 문제를 풀어 보면서
다이나믹 프로그래밍이 작동하는 원리, 그리고 필요성을 한 번 파헤쳐 봅시다!



2

A. 피보나치 수 5

- ✓ 여러분이 이미 풀어보신 문제입니다. 푸는 방법은 여러 가지이지만, 기본적으로 재귀를 이용하면 풀 수 있습니다. 피보나치 수를 구하는 함수를 `fibonacci(n)`이라 한다면, 아래와 같이 함수가 실행되며 뺏어나가게 됩니다.



2 A. 피보나치 수 5

- ✓ 코드로 나타낸다면, 아래와 같이 나타낼 수 있겠죠. 이 문제는 재귀 함수를 사용하는 선에서 맞출 수 있는 문제입니다. 여기까진 뭐 좋습니다.

```
#include <iostream>
using namespace std;

int get_fibonacci(int num) {
    if (num == 0) return 0;
    if (num == 1) return 1;

    return get_fibonacci(num - 1) + get_fibonacci(num - 2);
}

int main() {
    int N;
    cin >> N;

    cout << get_fibonacci(N);
}
```



B. 피보나치 수 2



- ✓ 그렇다면 이 문제는 어떨까요? 피보나치의 n 번째 수를 구하는 것은 동일하나 제한조건이 $0 \leq n \leq 20$ 에서 $0 \leq n \leq 90$ 로 바뀌었습니다.
- ✓ 이번에는 시간 초과를 받게 됩니다. 기존 방식이 어느 면에서 비효율적일 수 있을지 다음 페이지로 넘기기 전 한 번 고민해 보세요.

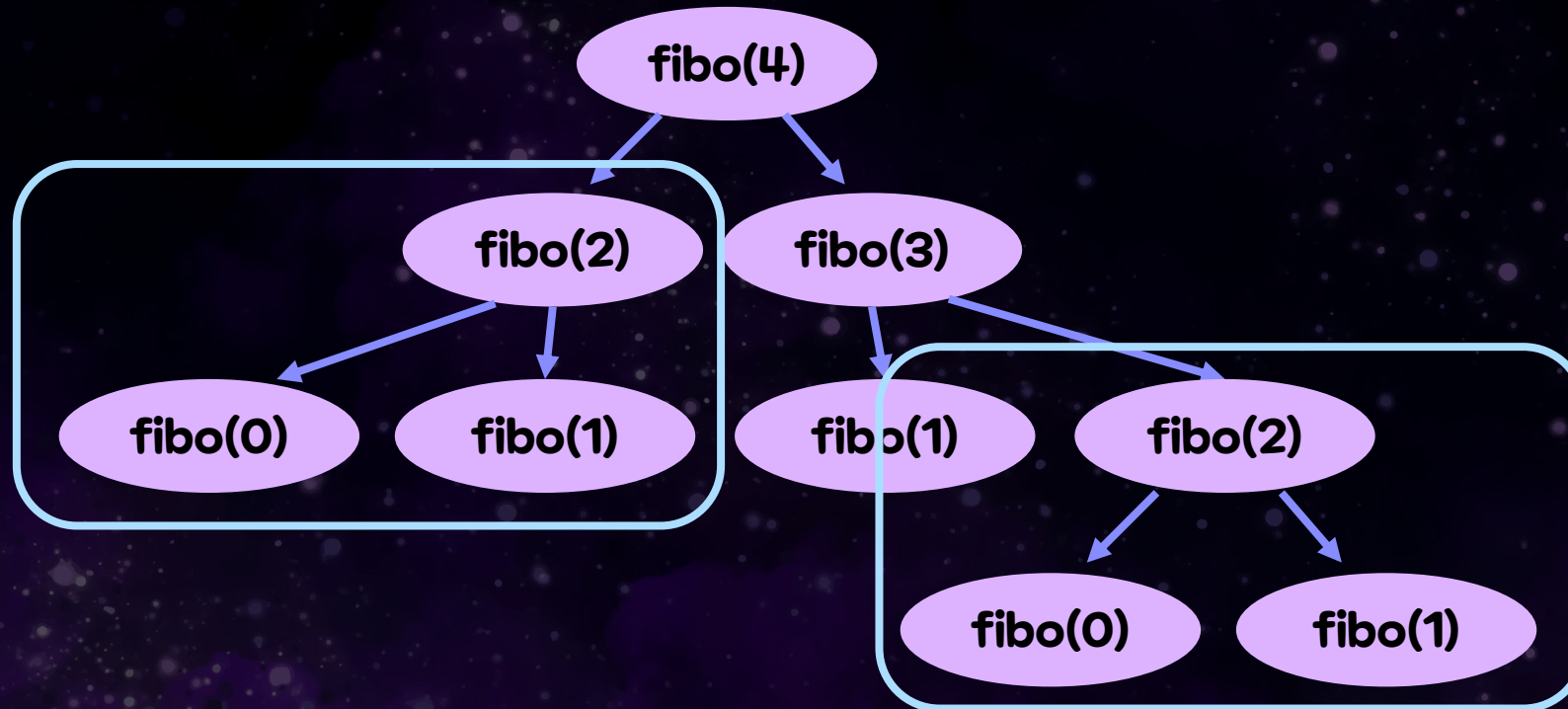
5 10826

시간 초과

1

B. 피보나치 수 2

- ✓ 핵심은 바로 재귀 과정 중 같은 파라미터의 함수가 여러 번 호출되어, 같은 연산을 여러 번 해야 하는 것에 있습니다. 그림을 보시면, `fibonacci(4)`를 구하는 과정에서 `fibonacci(2)`가 두 번 호출됨을 알 수 있습니다. 즉 불필요한 반복이 일어납니다.





B. 피보나치 수 2



- ✓ fibo(x)가 여러 번 호출되면, 이전 단계의 재귀함수도 여러 번 호출되게 되며, 이 단계가 지속될수록 함수 호출 횟수가 연쇄적으로 증가하게 됩니다.
- ✓ $n = 40$ 만 되어도 fibo(0)과 fibo(1)은 각각 63,245,986번, 102,334,155번 호출됩니다. $n = 10,000$ 이면 이 실행 횟수는 감도 잡을 수 없을 정도로 크겠죠.
- ✓ x번째 피보나치 수를 구하기 위해서는 fibo(x)를 한 번 실행하는 것으로 충분할 것입니다. 굳이 많은 횟수의 fibo(x)를 다시 호출해 무거운 재귀 연산을 다시 할 필요가 없습니다.
- ✓ 처음 fibo(x)를 구할 때만 직접 구하고, 그 결과를 저장해 둔 다음, 다시 fibo(x)가 호출된 경우에는 하위 단계로의 재귀를 진행하지 않고 바로 미리 저장해 둔 값을 반환한다면 이 연쇄적인 반복을 줄일 수 있을 것입니다.



B. 피보나치 수 2



- ✓ 다이나믹 프로그래밍은 탑-다운(Top-down), 바텀-업(Bottom-up)의 두 가지 방식으로 구현할 수 있습니다.
- ✓ 두 방식은 문제를 생각하고 값을 어떻게 채워나갈지의 초점이 다르나, 결과는 같습니다.
- ✓ 각각의 방법을 사용하여 이 문제를 해결해 봅시다.

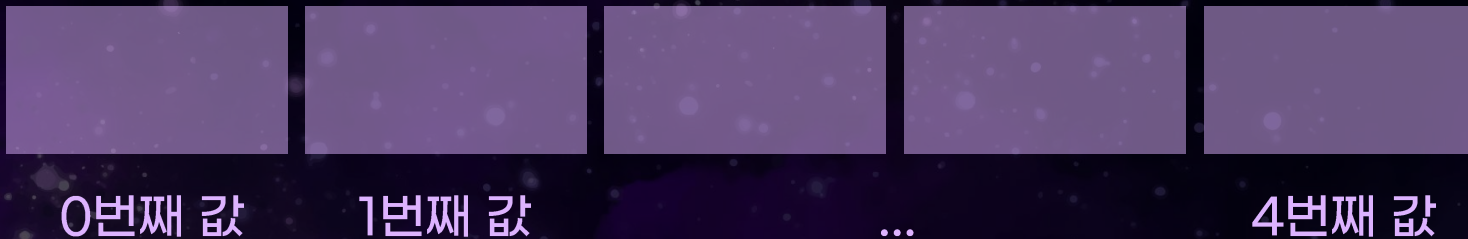
DP를 구현하는 방법: 탑다운 ✨

- ✓ 탑-다운은 큰 문제(top)의 해결을 초점으로 둡니다. 큰 문제는 당장 해결할 수 없지만, 큰 문제를 해결하는 데 필요한 작은 문제들을 정의하고, 그 작은 문제들을 다시 초점으로 뒤가며 단계를 뺏어내립니다(down).
- ✓ 탑-다운을 이용한 구현은 재귀를 사용합니다. 사실 피보나치 수를 구하는 문제를 이전 방식인 재귀를 이용하여 푸셨을 때와 거의 같습니다. 그래서 입문하는 과정에서 익숙한 방법이기도 합니다. 다른 점은 이미 해결한 문제는 dp 테이블에 값을 저장해 메모이제이션해 두고, 이후부터는 재사용한다는 점입니다.
- ✓ 다음 페이지에서 예를 들어봅시다.

DP를 구현하는 방법: 탑다운 ✨

- ✓ 가장 먼저, dp 테이블을 만들겠습니다. dp 테이블에 어떤 값을 채울지는 문제마다 다릅니다. 우리의 목적은 이미 구했던 피보나치 수를 또다시 구하지 않는 것이니, $dp[i] = i$ 번째 피보나치 수의 값이라고 정의해 봅시다.
- ✓ 초기에는 각 dp 테이블의 값이 비어있음을 나타내야 합니다. 이 값은 dp 테이블의 값으로 들어갈 일이 없는 값으로 초기화합니다. i 번째 피보나치 수의 값은 음수가 될 일은 없으니, -1 을 추천드려 봅니다.

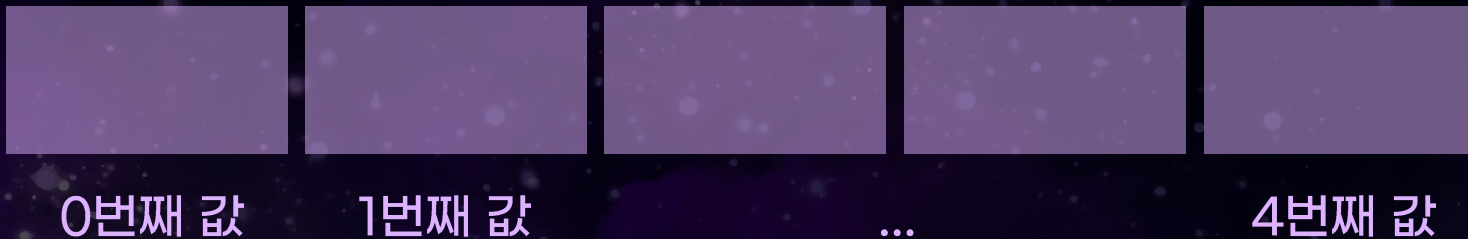
dp 테이블



DP를 구현하는 방법: 탑다운 ✨

- ✓ 또한, 작은 문제들을 토대로 큰 문제를 해결할 수 있으려면, 작은 문제와 큰 문제 간의 관계를 알아야 합니다. 이를 식으로 나타낸 점화식 또한 필요합니다.
- ✓ n 번째 피보나치 수는 $n - 1$ 번째 피보나치 수와 $n - 2$ 번째 피보나치 수의 합이므로, 점화식은 $dp[i] = dp[i - 1] + dp[i - 2]$ 가 됩니다.
- ✓ dp 테이블의 값을 어떻게 구성할지, 그리고 점화식을 통해 어떻게 채울지를 설계하는 것은 다이나믹 프로그래밍 문제의 공통 절차입니다.

dp 테이블

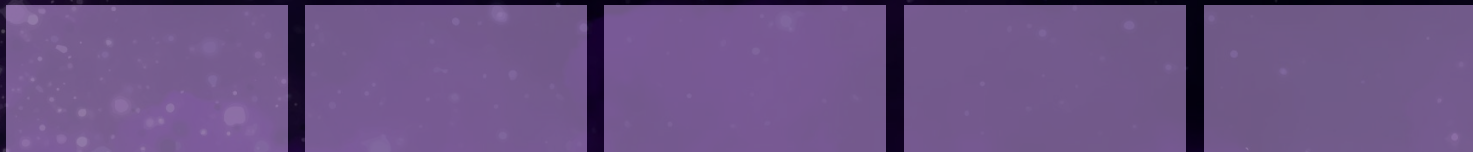


DP를 구현하는 방법: 탑다운 ✨

- ✓ 이제, 가장 큰 문제, $n = 4$ 일 경우의 피보나치 수를 구해 봅시다.

fibonacci(4)

dp 테이블



0번째 값

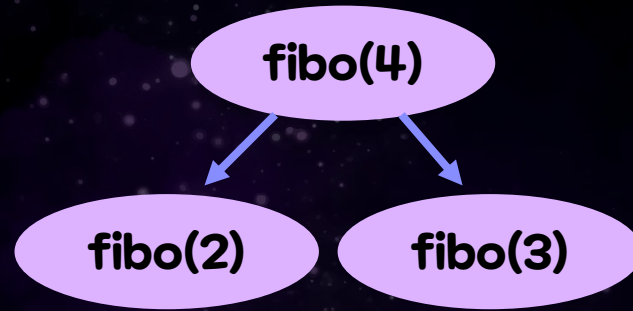
1번째 값

...

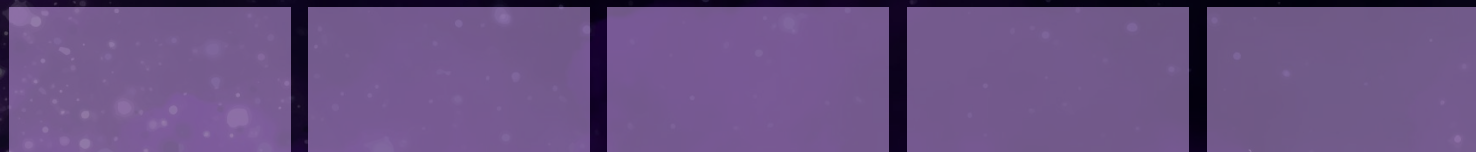
4번째 값

DP를 구현하는 방법: 탑다운 ✨

- ✓ 4번째 피보나치 수의 값은 당장 모르나 2번째 피보나치 수와 3번째 피보나치 수의 합임은 알 수 있습니다. 2, 3번째 피보나치 수 또한 이런 방식으로 더 작은 문제로 분할할 수 있습니다.



dp 테이블



0번째 값

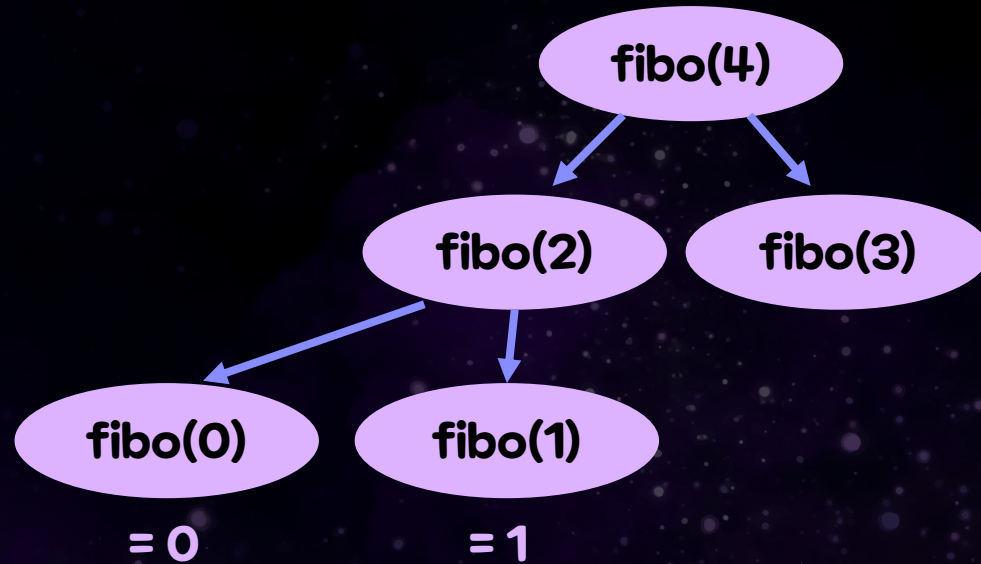
1번째 값

...

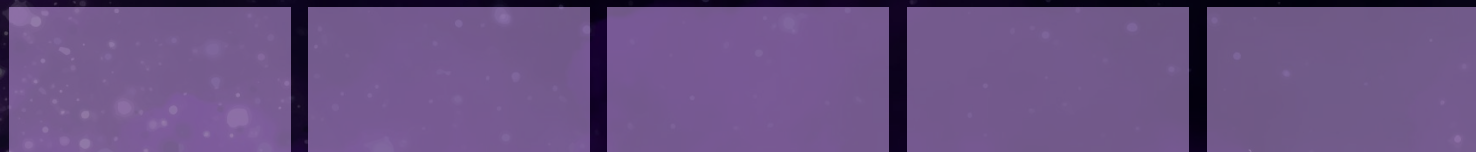
4번째 값

DP를 구현하는 방법: 탑다운 ✨

- ✓ 이 방식으로 더 진행해 봅시다. 이제 2번째 피보나치 수의 값을 구하기 위한 모든 값들이 준비되었습니다.



dp 테이블



0번째 값

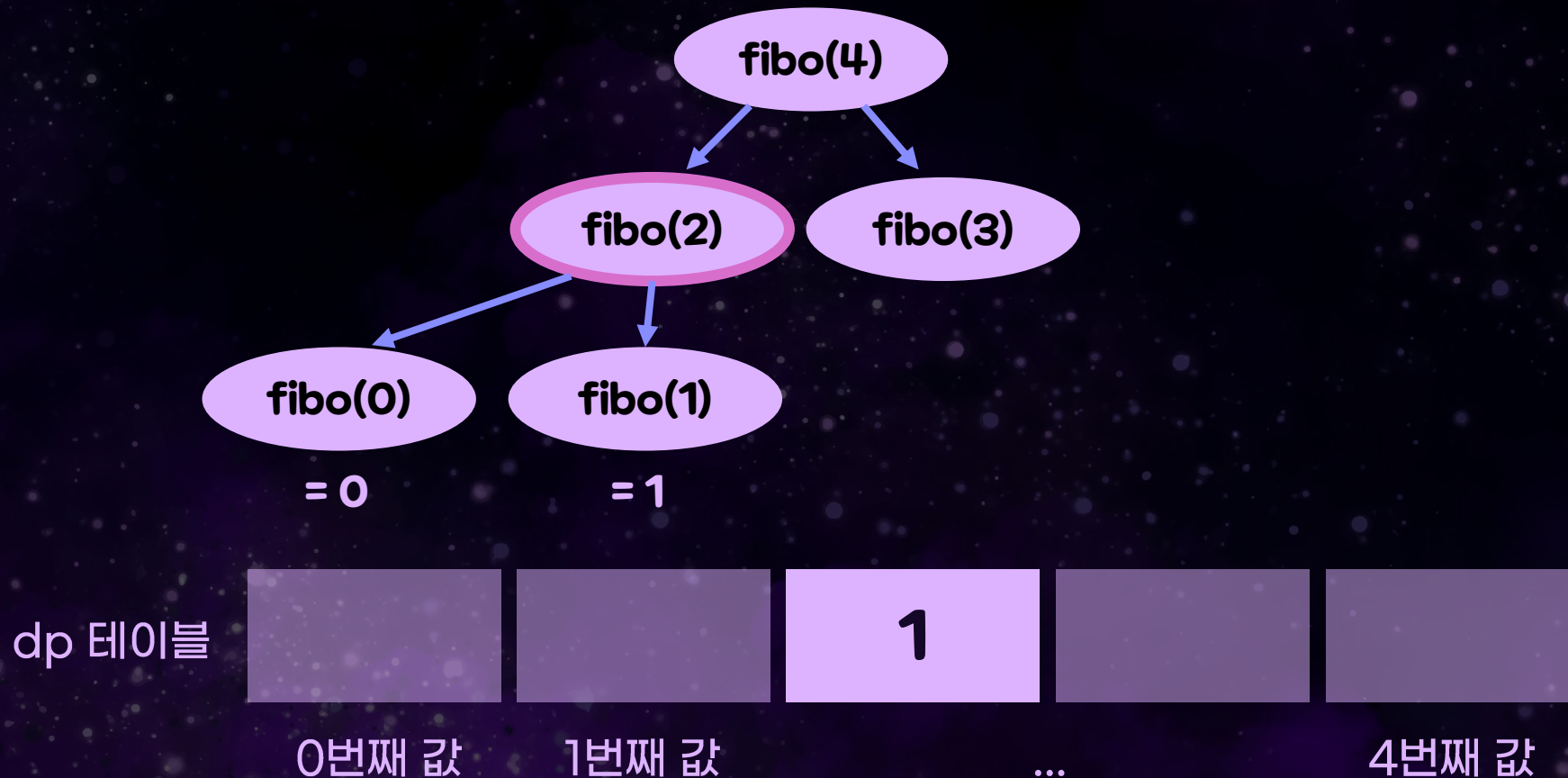
1번째 값

...

4번째 값

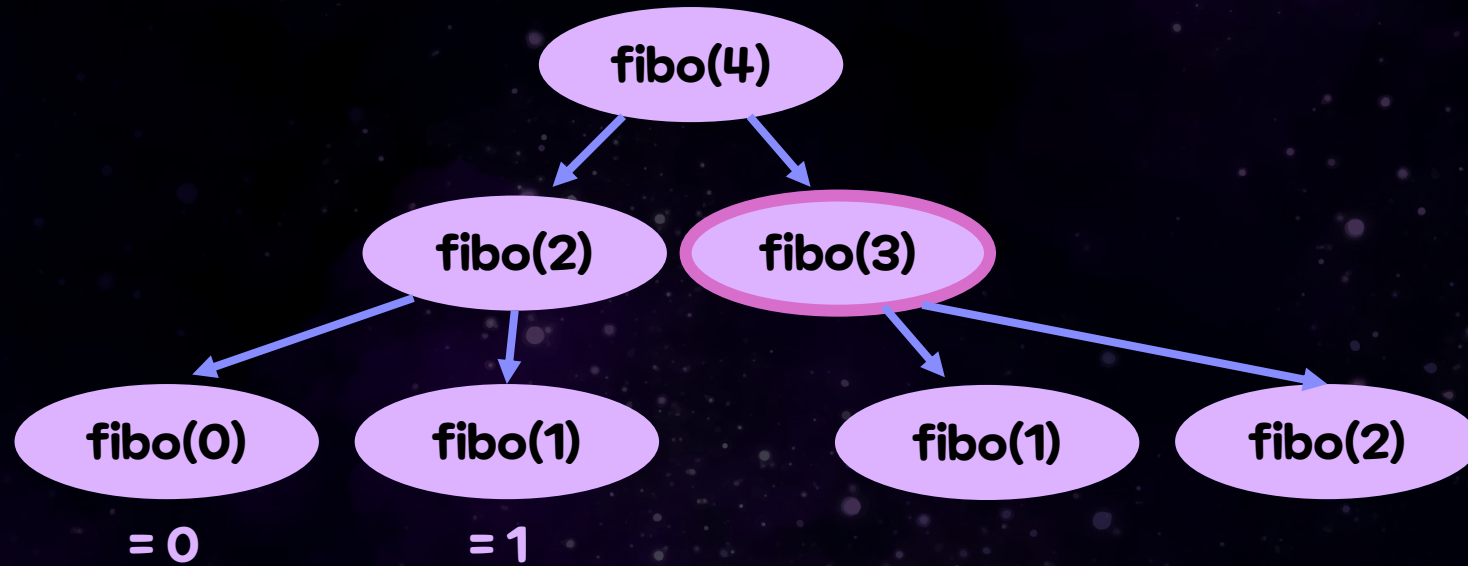
DP를 구현하는 방법: 탑다운 ✨

- ✓ 점화식에 의해 2번째 피보나치 수의 값은 $0 + 1 = 1$ 임을 알 수 있습니다. 재귀라면 `fibonacci(2)`에서 1을 리턴하는 것으로 끝났지만, 이번엔 기록도 해봅시다. `dp[2] = 1`로요.



DP를 구현하는 방법: 탑다운 ✨

✓ 이제 `fibonacci(3)`에서 아래로 뺏어내려가 볼까요?



dp 테이블



0번째 값

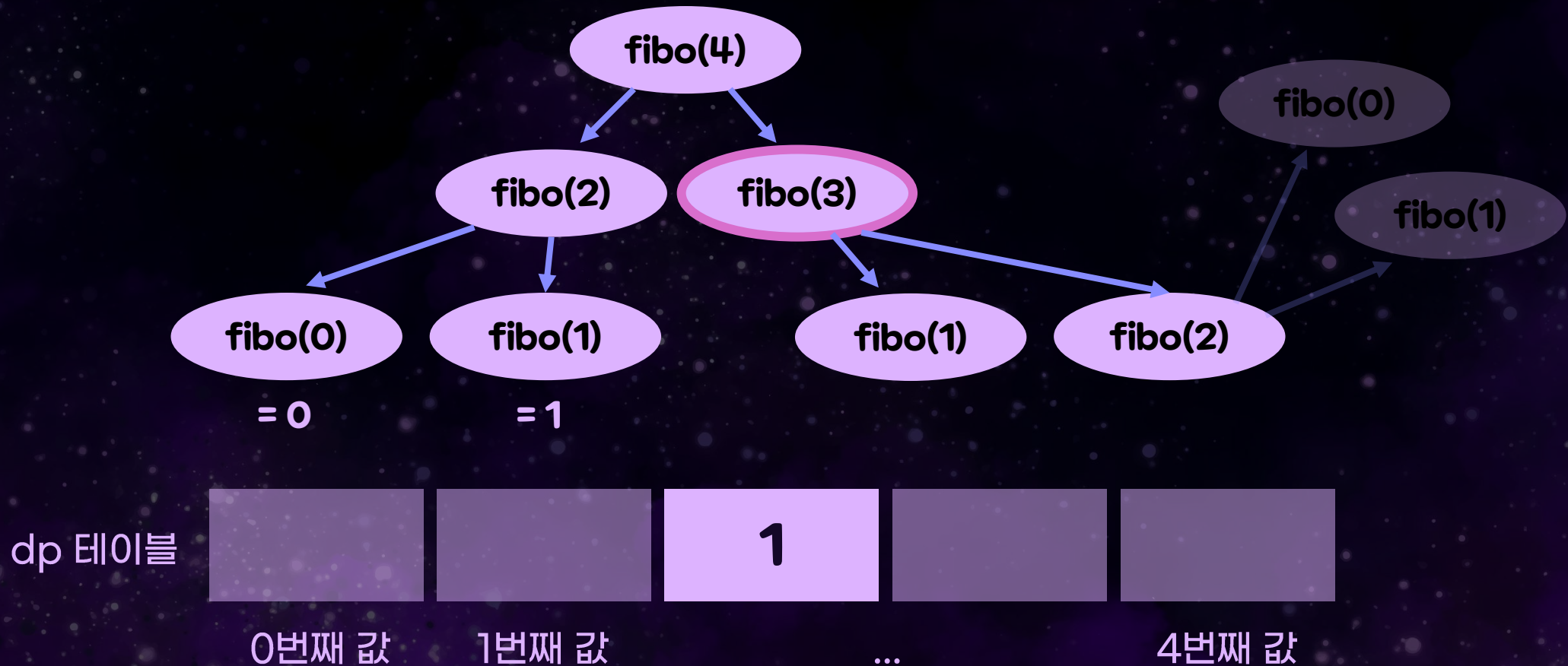
1번째 값

...

4번째 값

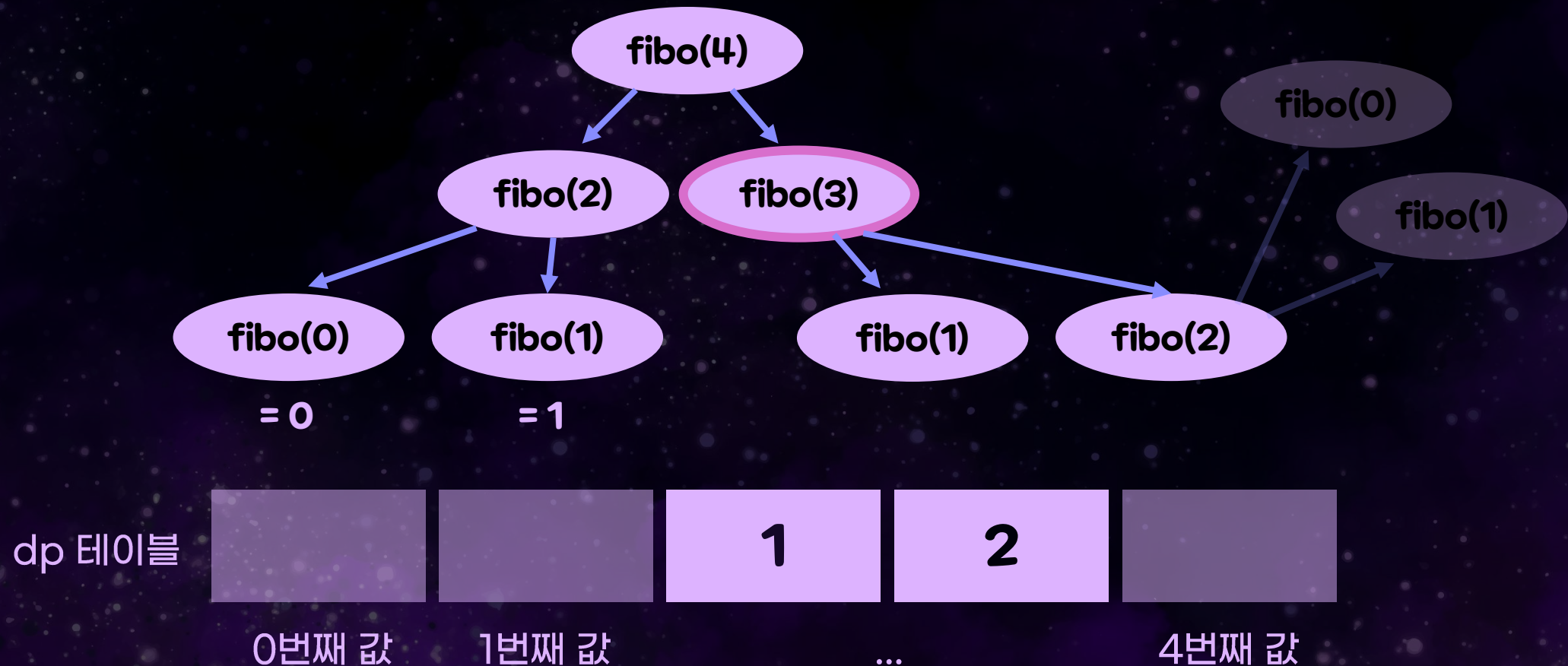
DP를 구현하는 방법: 탑다운 ✨

- ✓ 원래라면 `fibonacci(2)`를 호출해 다시 뺏어내려가야 하지만, `dp[2]`에 이미 저장된 값이 있습니다! 정의대로면 `dp[2]`는 2번째 피보나치 수이므로, 이전과 달리 즉시 답을 구할 수 있습니다.



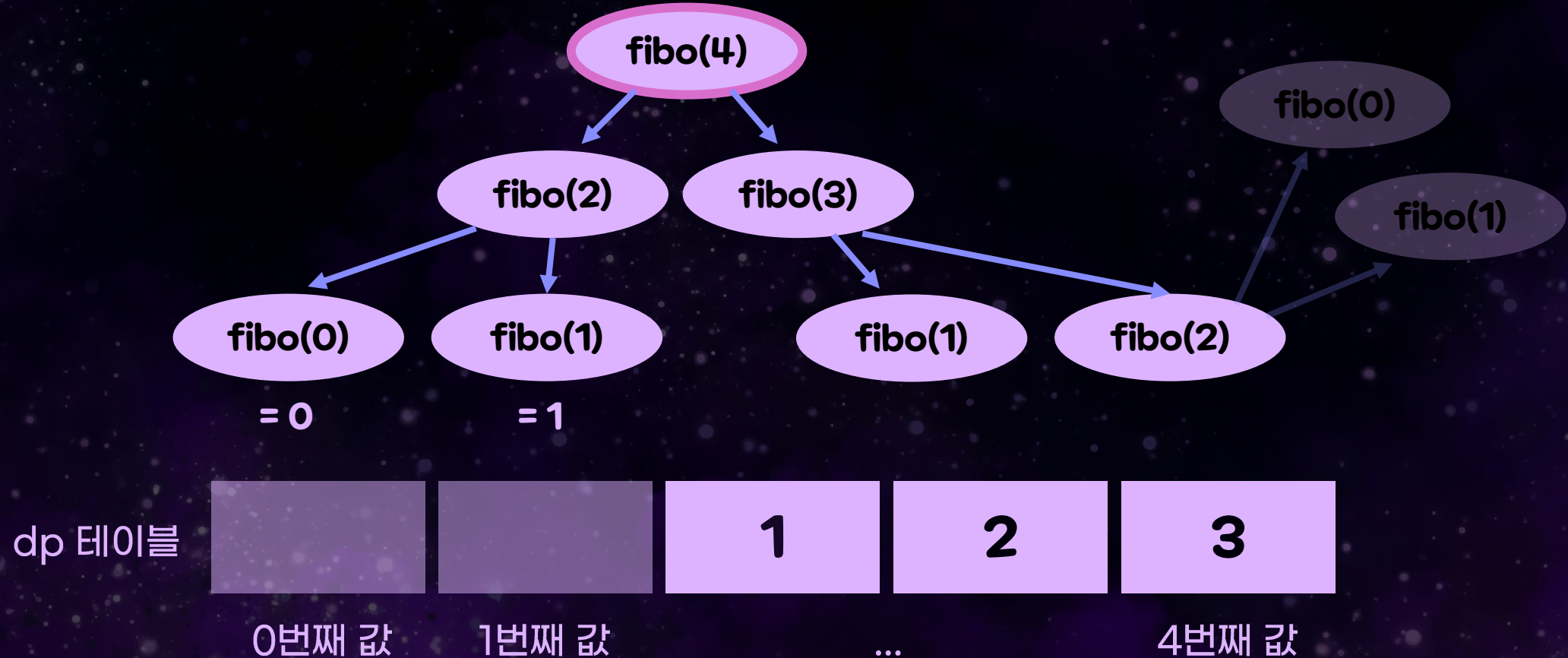
DP를 구현하는 방법: 탑다운 ✨

- ✓ 물론, `fibonacci(3)` 역시 값을 반환하기 전 답을 dp 테이블에 기록하고요!



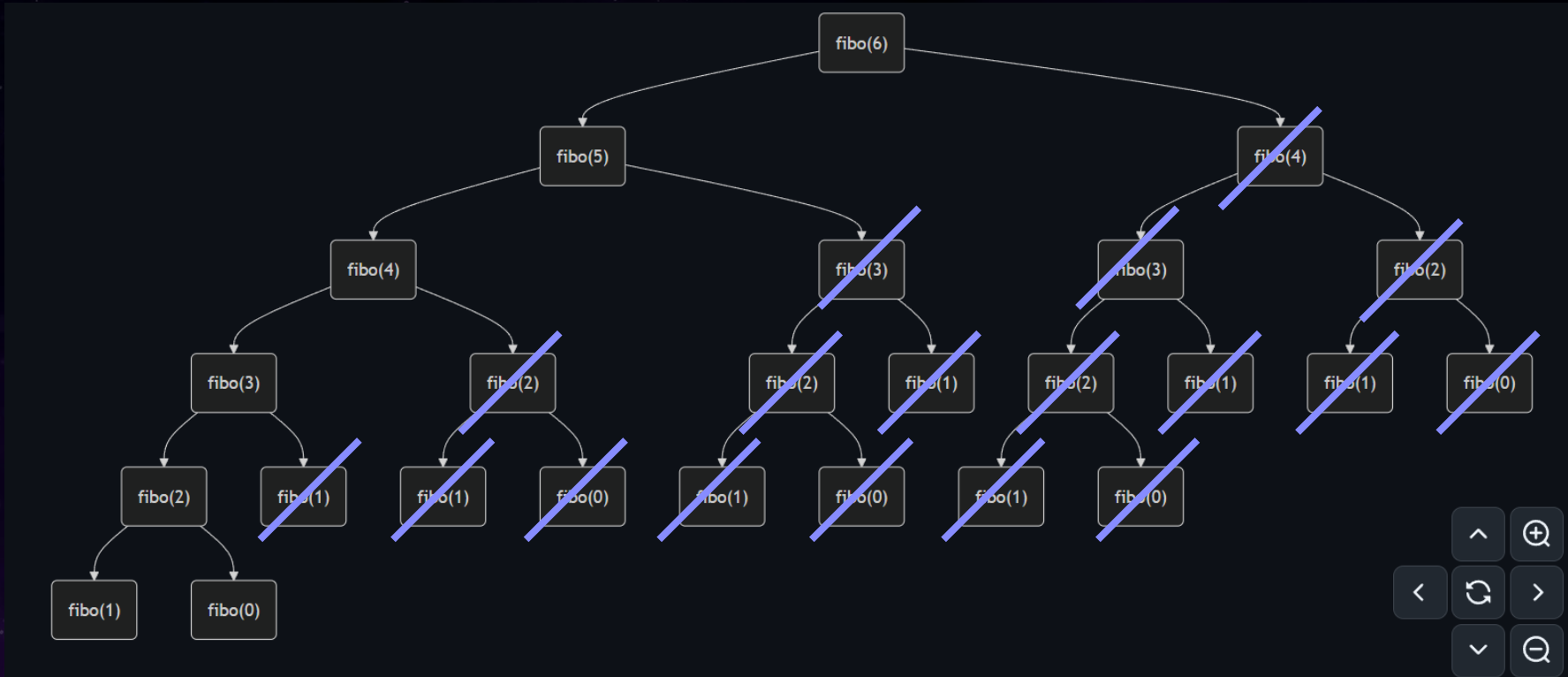
DP를 구현하는 방법: 탑다운 ✨

- ✓ 이렇게 하면 최종적으로 4번째 피보나치 수의 값도 구할 수 있게 됩니다.



DP를 구현하는 방법: 탑다운 ✨

- ✓ 보너스로, $n = 6$ 일 경우에서 탑-다운 dp를 통해 반복을 줄인 예시도 보여드릴게요. n 이 커짐에 따라 이 차이는 커지겠죠? 필요성을 체감해 보시면 좋을 것 같아요.



DP를 구현하는 방법: 탑다운 ✨

✓ 마지막으로, 탑-다운을 이용해 피보나치 수를 구하는 코드의 예시입니다(C++).

dp 테이블 선언부입니다.

dp 테이블에 이미 값이 채워져 있으면, 재귀를 통해 답을 다시 구할 필요가 없습니다. 즉시 답을 반환해 줍시다.

초기값은 이미 우리가 알고 있으니, 그대로 반환해 줍시다.

그 외에는 평소처럼 재귀를 이용해 값을 계산하되, 반환 전 dp 테이블에 값을 저장해 둡시다.

-1을 초기값으로 dp 테이블을 초기화했습니다. 이렇게 하면 dp 테이블에 계산된 값이 담겼는지를 구분할 수 있습니다. 설명드렸듯이 꼭 -1일 필요는 없습니다. 구별만 되면 어떤 값이든 좋습니다.

```
#include <iostream>
#include <algorithm>
typedef long long ll;
using namespace std;

ll dp[91];

ll get_fibonacci(int num) {
    if (dp[num] != -1) return dp[num];

    if (num == 0) return 0;
    if (num == 1) return 1;

    dp[num] = get_fibonacci(num - 1) + get_fibonacci(num - 2);
    return dp[num];
}

int main() {
    int N;
    cin >> N;

    fill(dp, dp + N + 1, -1);
    cout << get_fibonacci(N);
}
```

DP를 구현하는 방법: 바텀업 ✨

- ✓ 바텀-업은 작은 문제(bottom)의 해결을 초점으로 둡니다. 작은 문제들을 해결하면 이를 이용해 큰 문제를 해결할 수 있기 때문입니다. 점점 높은 단계로 값을 끌어올립니다(up).
- ✓ 바텀-업을 이용한 구현은 주로 반복문을 사용합니다. 구현이 대체로 깔끔해 구현하기 좋습니다. 다만 경험상 복잡하거나 어려운 문제를 풀 때는 구현 난이도가 높게 느껴지기도 했습니다.
- ✓ 이번에도 다음 페이지에서 예를 들어봅시다.

DP를 구현하는 방법: 바텀업 ✨

- ✓ 가장 먼저, dp 테이블을 만들겠습니다. $dp[i]$ = i 번째 피보나치 수의 값이라고 똑같이 정의하겠습니다.
- ✓ 여기에 $dp[0] = 0$, $dp[1] = 1$ 로 초기값을 설정하겠습니다. 초기값이 있어야 이 값들을 토대로 점화식을 이용해 다른 값들을 채워나갈 수 있기 때문입니다. 탑다운의 경우 재귀 함수 내에 따로 분기처리를 둔 바 있었습니다.
- ✓ 초기값은 점화식을 이용해 전개해 나가기 편한 선에서 자유롭게 채워주시면 됩니다.



DP를 구현하는 방법: 바텀업 ✨

- ✓ 이제, 값을 채우지 않은 2번째 값부터 점화식을 이용해 채우겠습니다. $i = 2$ 부터 반복문을 돌 것입니다.
- ✓ 매 i 마다 $dp[i]$ 의 값을 $dp[i - 1] + dp[i - 2]$ 를 설정해 줄 것입니다. 정확히 점화식을 활용한 예시죠? 그러면 $dp[2] = dp[0] + dp[1]$ 이므로, 1이 채워집니다.



DP를 구현하는 방법: 바텀업 ✨

- ✓ $dp[3]$, $dp[4]$ 또한 반복문을 돌며 같은 방법으로 채울 수 있습니다. 이렇게 해서 4번째 피보나치 수의 값을 구했습니다! 이게 끝입니다. 비교적 간단하죠?



DP를 구현하는 방법: 바텀업 ✨

✓ 바텀-업을 이용해 피보나치 수를 구하는 코드의 예시입니다(C++).

dp 테이블 선언부입니다.

dp 테이블에 이미 값이 채워져 있으면, 재귀를 통해 답을 다시 구할 필요가 없습니다. 즉시 답을 반환해 줍시다.

초기값들을 설정해 주는 부분입니다.

나머지 부분들은 이미 채운 dp 값들과 점화식을 이용해, 반복문을 이용해 큰 문제를 점차 해결해 나갈 수 있습니다.

```
#include <iostream>
typedef long long ll;
using namespace std;

ll dp[91];

int main() {
    int N;
    cin >> N;

    dp[0] = 0;
    dp[1] = 1;

    for (int i = 2; i <= N; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }

    cout << dp[N];
}
```

시간복잡도를 정리한다면 ✨

- ✓ 단순 재귀로 피보나치 수를 구할 때의 시간복잡도는 $O(2^n)$ 이지만, 다이나믹 프로그래밍을 활용했을 때의 시간복잡도는 $O(n)$ 이 됩니다. dp 테이블의 각 값을 채우는 데 $O(1)$ 의 시간복잡도가 들고, dp 테이블의 값 개수만큼만 과정을 채우게 되기 때문입니다.
- ✓ 두 문제를 풀어보시면서 효율성의 체감을 느끼셨을 것이라 생각합니다.

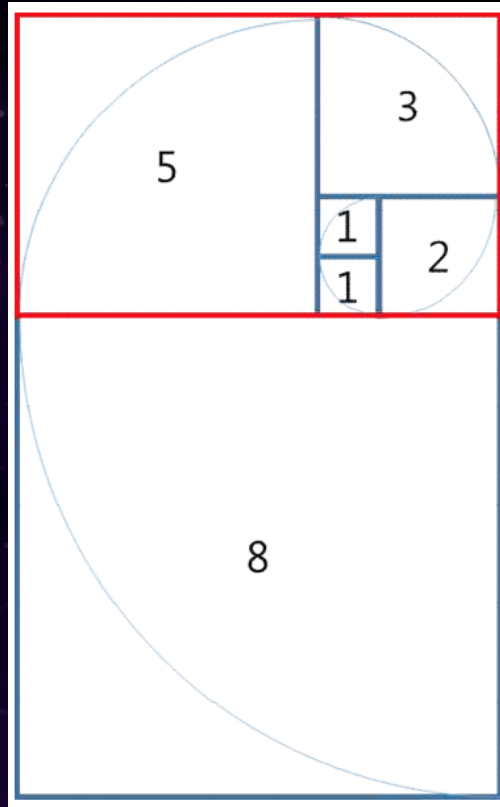
마무리 ✨

- ✓ 이로서 두 문제를 풀어보면서 다이나믹 프로그래밍의 필요성 및 구현 방법을 익혔습니다. 느끼셨겠지만 어떻게 문제를 하나의 큰 문제와 부분 문제들로 보고 상황을 정의할 수 있을지, 점화식은 어떻게 세울 수 있을지 등등 모두 발상이자 설계의 영역입니다. 다양한 문제를 연습해 보면서 실력을 향상시켜 보시기 바랍니다.
- ✓ 탑-다운과 바텀-업 두 방법 모두 사용하기 좋은 방식이며, 편하신 방법을 택하시면 되겠습니다. 일부 문제들은 이 중 한쪽을 택하는 것이 더 편하게 문제를 정리할 수 있기도 해, 스스로의 판단이 필요한 영역입니다.
- ✓ 다음 페이지부터는 남은 문제들에 대한 풀이를 설명하겠습니다. 문제를 관찰하시고, 어떻게 문제를 해결할 수 있을지 고민해 보세요!

5

C. 타일 장식물

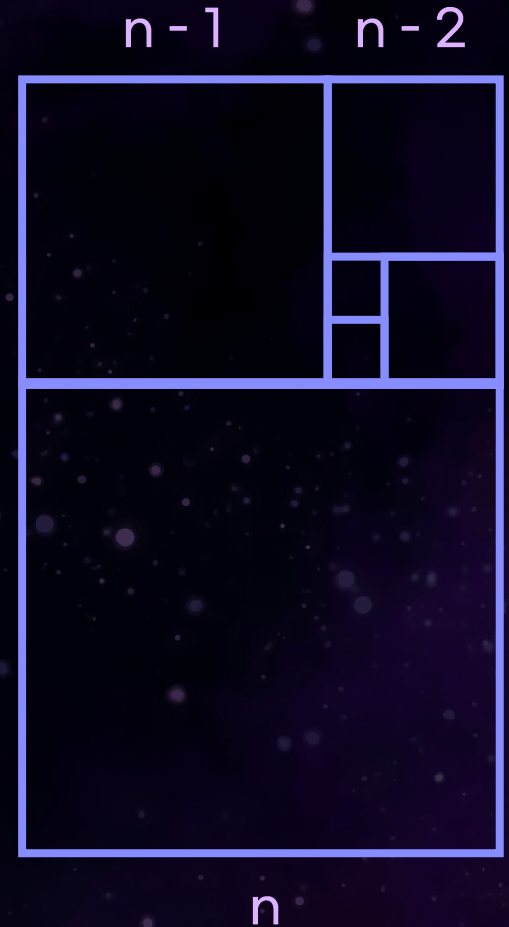
- ✓ 타일의 크기가 단계가 높아짐에 따라 길이가 어떤 규칙으로 증가하는지를 알아내는 것이 관건입니다. n단계 타일의 길이에 초점을 둔다면...?



5

C. 타일 장식물

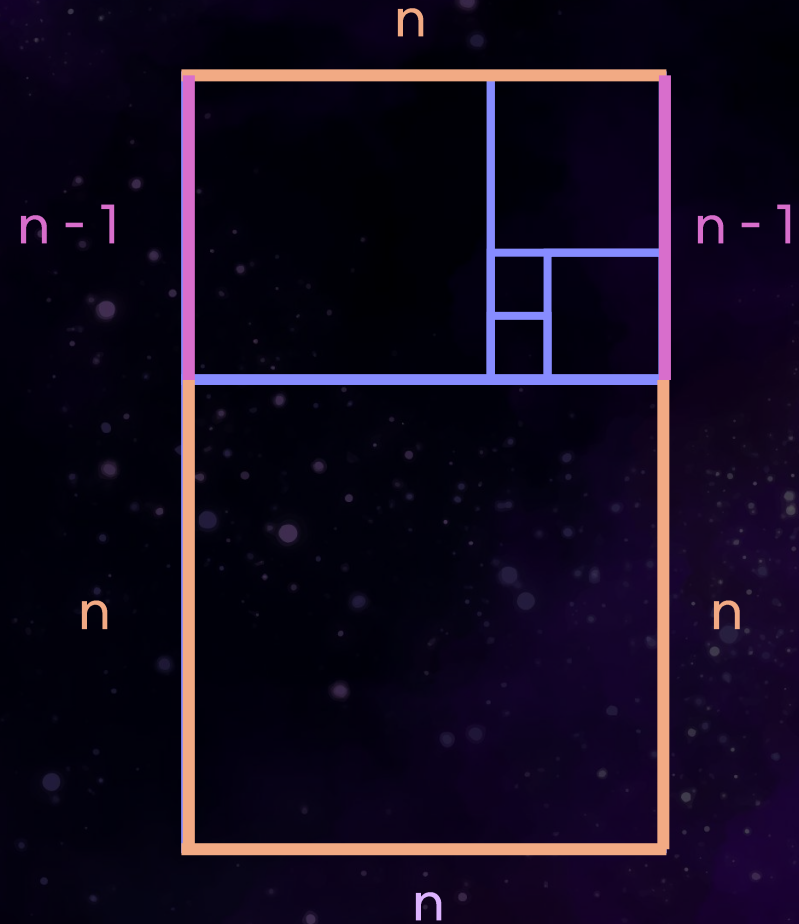
- ✓ 1단계, 2단계 타일의 한 변의 길이는 1입니다.
- ✓ 그리고 n 단계 타일의 한 변의 길이는 $n - 1$ 단계 타일과 $n - 2$ 단계 타일의 한 변의 길이의 합과 같습니다. 피보나치 수열의 점화식과 같다는 것을 알아낼 수 있습니다.
- ✓ $dp[i] = i$ 번째 타일의 한 변의 길이로 정의하고, $dp[i] = dp[i - 1] + dp[i - 2]$ 를 점화식으로 둔다면 n 번째 타일의 한 변의 길이를 구할 수 있습니다.



1, 1, 2, 3, 5, 8, ...

피보나치 수열인 게 너무 티가 나긴 합니다...

- ✓ 구해야 하는 것은 타일 장식물 전체의 둘레입니다. 길이를 이용하면 답을 구하는 것이 가능합니다.
- ✓ 우측 그림을 참고하시면 아시겠지만, n 번째 전체 직사각형의 둘레는 $n - 1$ 번째와 n 번째 타일의 한 변의 길이를 조합하여 구할 수 있음을 알 수 있습니다. 식으로 정리하면 답은 $dp[n] \times 4 + dp[n - 1] \times 2$ 가 됨을 알 수 있습니다.
- ✓ 종합적으로 규칙을 파악해 타일 장식물의 길이와 둘레를 각각 구하는 방법을 식으로 표현하여 풀 수 있는 문제였습니다.





D. BABBA



- ✓ 제한조건이 최대 $n = 45$ 로 미묘한데요, 문제에서 요구하는 대로 직접 문자열을 만들어 시뮬레이션을 하면 시간 초과를 받는 문제입니다. n 의 값에 따라 문자열의 길이가 이전 단계의 두 배 수준으로 빠르게 증가하기 때문입니다. 실제로 $n = 45$ 인 경우 만들어지는 문자열의 길이는 **18억**에 달합니다.
- ✓ 따라서 문자열을 일일이 만들 수는 없해보입니다. 효율적으로 문제를 해결할 방법 없을까요?



D. BABBA



- ✓ 핵심은 바로 문제에서 요구하는 것은 오로지 “A”, “B”의 개수뿐이라는 것입니다. 어떤 문자열이 만들어지는지 요구하지 않는다는 것이죠! 즉 각 단계에서는 문자열 그 자체가 아닌 A의 개수, B의 개수만을 관리해도 충분할 것입니다.
- ✓ 각 값을 [A 개수, B 개수] 와 같이 관리하고, 이전 단계와의 관계를 유추해 봅시다.



D. BABBA



- ✓ 버튼을 누를 때마다 화면의 모든 B는 BA로 바뀐다는 것은 현재 단계의 B 개수만큼 다음 단계의 A, B 개수를 이룬다는 뜻이 됩니다.
- ✓ 버튼을 누를 때마다 화면의 모든 A는 B로 바뀐다는 것은 현재 단계의 A 개수만큼 다음 단계의 B 개수를 이룬다는 뜻이 됩니다.
- ✓ i단계에서의 A 개수, B 개수를 $a[i]$, $b[i]$ 라 하면,
 $a[i] = b[i - 1]$, $b[i] = a[i - 1] + b[i - 1]$ 이 되는 것이죠!
- ✓ 위의 식을 점화식으로 사용하여 A, B 개수를 각각 단계별로 관리하면 정답을 구할 수 있습니다.

E. 달나라 토끼를 위한 구매대금 지불 도우미 ✨

- ✓ 흔히 그리디 알고리즘으로 잘 알려진 거스름돈 문제와 동일하게 접근하면 풀리기 쉽습니다. 가장 큰 단위의 돈으로 먼저 거슬러 주고, 그 다음으로 큰 단위 돈으로 최대한 거슬러 주고, ... 를 반복하는 전략을 이야기하는 것입니다.
- ✓ 이는 큰 단위의 동전이 작은 단위의 동전의 배수라는 원칙이 깨졌기 때문입니다. 1원, 2원, 5원, 7원이라니, 완전 무근본 조합입니다. 작은 단위의 동전으로 거슬러 줄 수 있는 경우를 완벽하게 큰 단위의 동전으로 거슬러 주는 방식으로 항상 교체할 수 없으니 그리디한 전략은 통하지 않습니다.
- ✓ 어떻게 이 문제를 접근해 볼 수 있을까요?

E. 달나라 토끼를 위한 구매대금 지불 도우미 ✨

- ✓ “우리가 거슬러줘야 하는 전체 돈을 최소한의 동전 개수로 거슬러 주는 것” 을 하나의 큰 문제로 보고, 이를 위해 “그 돈의 일부를 최소한의 동전 개수로 거슬러 주는 것” 을 부분 문제로 봅시다. 부분 문제를 통해 전체 문제를 해결할 수 있는 방식으로 흘러간다면 다이나믹 프로그래밍을 이용한 접근이 가능할 것입니다.
- ✓ $dp[i]$ = i 원을 거슬러줄 수 있는 최소 동전 개수라고 정의도 해 봅시다. 이렇게 하면 부분 문제의 답을 저장하고 재사용하기 수월할 것입니다. 이렇게 하면 $dp[N]$ 이 답이 됩니다.

3

E. 달나라 토끼를 위한 구매대금 지불 도우미 ✨

- ✓ 우선, dp 테이블의 정의대로, 값들을 일부 채워봅시다.
- ✓ 편의상 바텀-업으로 풀어보겠습니다. 이 경우 반복문을 통해 8부터 N까지 돌면서 값을 채워야 합니다.
- ✓ 점화식을 어떻게 도출해볼 수 있을까요?

	0	1	1	2	2	1	2	1	
인덱스	0	1	2	3	4	5	6	7	8

$dp[i]$ = i원을 거슬러줄 수 있는 최소 동전 개수

3

E. 달나라 토끼를 위한 구매대금 지불 도우미 ✨

- ✓ 아직 구하지는 않았지만, 동전을 거슬러주는 최적의 경우가 있다고 가정해 봅시다. 이 경우 0원이 아닌 한 1개 이상의 동전으로 구성되어 있을 거예요. 그게 뭔지는 모르지만...



3

E. 달나라 토끼를 위한 구매대금 지불 도우미 ✨

- ✓ 그리고, 이 구성되어 있는 동전 중 마지막 동전은 무조건 1원, 2원, 5원, 7원 중 하나일 것입니다.



3

E. 달나라 토끼를 위한 구매대금 지불 도우미 ✨

- ✓ 여기에서, 마지막 동전의 액면가를 고정해 봅시다. n 원을 모으는 최적의 경우에서, 마지막 동전은 1원이라고 고정해 볼게요. 실제로는 아닐 수도 있지만 정해졌다고 해 볼게요.
- ✓ 그러면, 우리가 결정할 수 있는 동전은 1원을 뺀 나머지 $n - 1$ 원이 됩니다.
- ✓ 따라서 문제를 해결하는 방향은 나머지 $n - 1$ 원을 최소 개수의 동전을 이용해 거슬러 주는 방법을 찾는 것입니다.



3

E. 달나라 토끼를 위한 구매대금 지불 도우미 ✨

- ✓ 그런데 우리는 dp 테이블의 정의를 $dp[i] = i$ 원을 거슬러줄 수 있는 최소 동전 개수라고 정해졌었습니다.
- ✓ 그렇기 때문에 $n - 1$ 원을 거슬러줄 수 있는 최소 동전의 개수는 $dp[n - 1]$ 이 됩니다. 정의를 따르니까요.
- ✓ 1원을 마지막 동전으로 고정시키는 경우, 점화식은 $dp[n] = dp[n - 1] + 1$ 이 되는 것입니다.

$$dp[n] = \text{?원} \quad \text{?원} \quad \text{?원} \quad \dots \quad \text{?원} \quad \text{1원}$$
$$dp[n] = dp[n - 1] + 1$$

E. 달나라 토끼를 위한 구매대금 지불 도우미 ✨

- ✓ 이제 실제 문제로 확장해봅시다. 최적해의 마지막 동전의 액면가는 사실 1원, 2원, 5원, 7원 중 하나가 될 수 있습니다.
- ✓ 따라서, 우리는 이 네 가지 경우를 모두 따져본 후, 이 중 가장 적은 개수의 동전을 사용하는 방법을 답으로 택할 것입니다.
- ✓ 점화식은 $dp[i] = \min(dp[i - 1], dp[i - 2], dp[i - 5], dp[i - 7]) + 1$ 이 되죠.
- ✓ 이렇게 해서 구한 값은 i 원을 거슬러주기 위한 최소 동전 개수에 부합하므로, 추후 이 값을 저장해 두었다가 이후의 해들을 구할 때 다시 사용할 수 있을 것입니다.
- ✓ 생소하게 느끼실 수도 있지만, 그럴수록 dp 테이블의 정의를 토대로 생각해보시면 이해에 도움이 되실 거라 생각합니다. 정의가 정말 중요합니다.



The End